Computer Architecture

Chapter 8
Pipelining

Afarghadan@aut.ac.ir

عظیم فرقدان

اسفند ۱۴۰۳

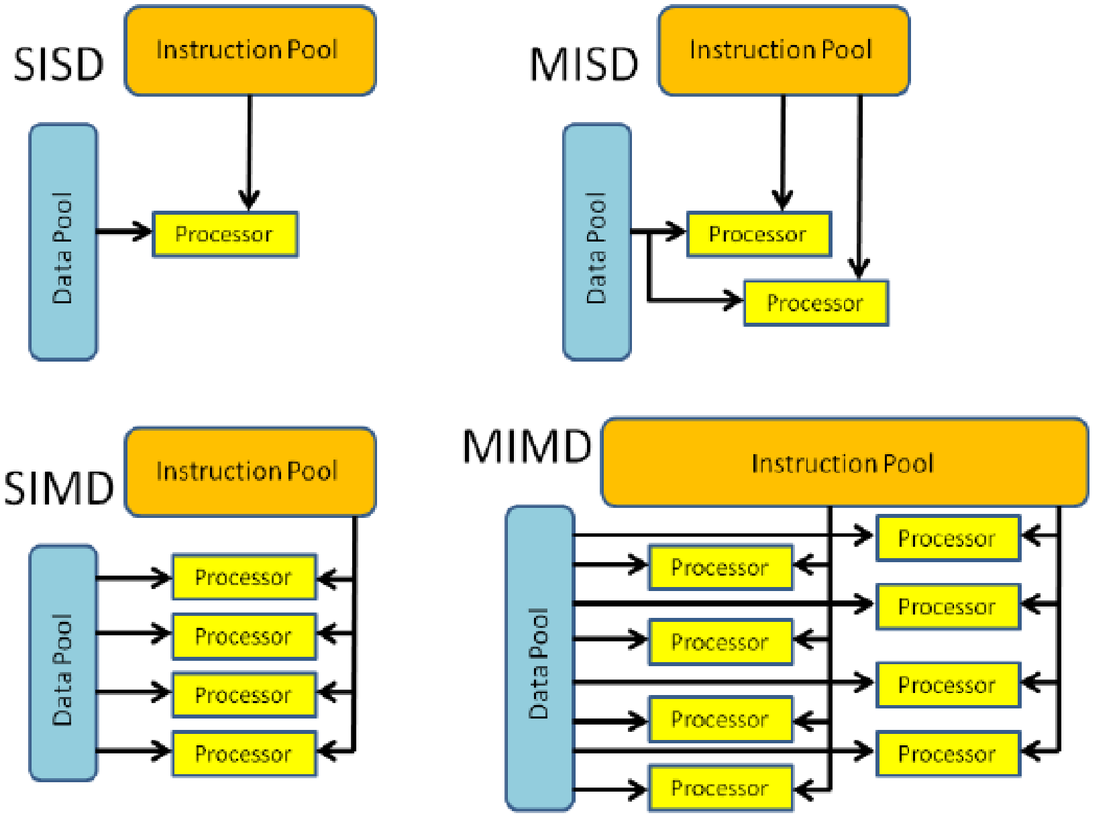
Lecture Overview

1. **Parallel Processing**
2. **Pipelining**
3. **Arithmetic Pipeline**
4. **Instruction Pipeline**
5. **Pipeline Conflicts**

Parallel Processing

- Parallel processing may occur in the instruction stream, in the data stream, or in both:

- **Single** Instruction Stream, **single** data stream (SISD)
- **Single** Instruction Stream, **multiple** data stream (SIMD)
- **multiple** Instruction Stream, **single** data stream (MISD)
- **multiple** Instruction Stream, **multiple** data stream (MIMD)



Speedup and Efficiency

Speedup is a performance metric used in parallel computing and optimization. It shows how much faster a program runs when using a faster or more efficient system (e.g., parallel processing) compared to a baseline system (usually serial execution).

$$\text{Speedup} = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$$

Efficiency is a measure of how effectively the available processors are being used in a parallel system. It shows the proportion of time the processors are doing useful work (as opposed to being idle or waiting).

$$\text{Efficiency} = \frac{\text{Speedup}}{p}$$

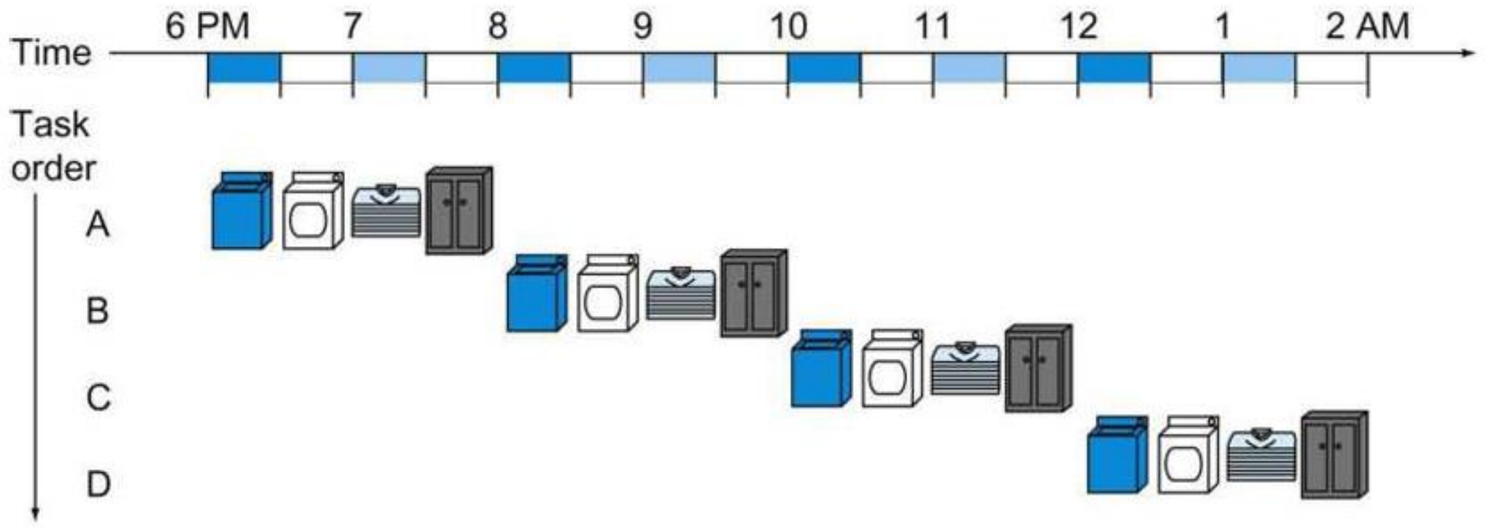
Example

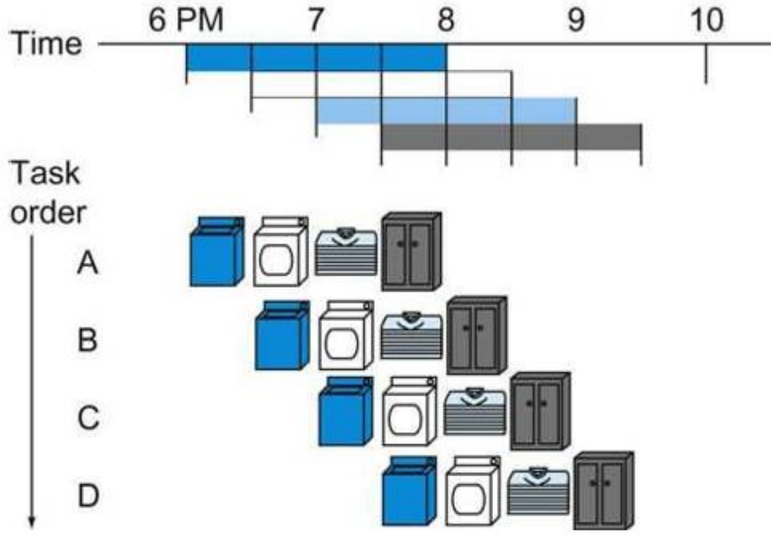
In a system with parallel architecture, assuming there are *p* processors and *f* percent of the instructions cannot be parallelized, what is the maximum possible speedup?

Now, if *f* is equal to 20 percent, calculate the maximum speedup.

Parallel Processing

- Techniques that provide **simultaneous** data-processing tasks that results in **faster** execution time and increase in **throughput**.
- Pipelining is an **implementation technique** in which multiple instructions are **overlapped** in **execution**.

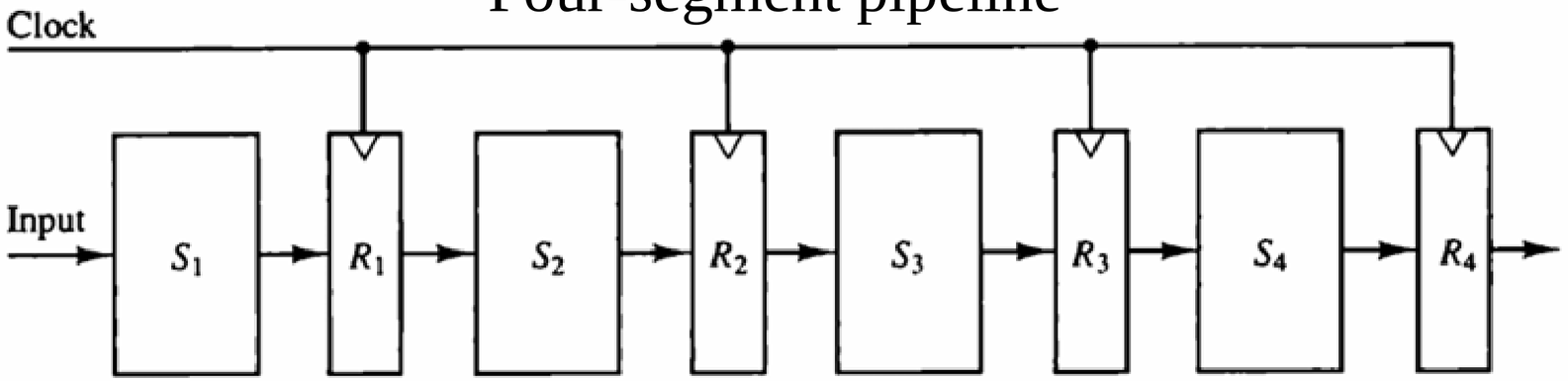




Pipelining

- Any operation that can be **decomposed into a sequence of sub operations** of **about the same complexity** can be implemented by a pipeline processor.

Four-segment pipeline



Segment:	1	2	3	4	5	6	7	8	9
1	T_1	T_2	T_3	T_4	T_5	T_6			
2		T_1	T_2	T_3	T_4	T_5	T_6		
3			T_1	T_2	T_3	T_4	T_5	T_6	
4				T_1	T_2	T_3	T_4	T_5	T_6

Clock cycles

Pipelining

- A k-segment pipeline with a clock cycle time t_p is used to execute n tasks:

Time required for the first task:	$\mathrm{K}t_p$
Next n-1 tasks are emerge one task per clock:	$(\mathrm{n}-1)t_p$
Total clock cycles required:	$\mathrm{k}+(\mathrm{n}-1)$

- A nonpipeline unit performs the same operation and takes a time of t_n to complete each task.

Total time required for n tasks:	$\mathrm{n}t_n$

Pipelining

- **Pipeline speedup:**

$$S = \frac{nt_n}{(k+n-1)t_p}$$

- Increase in number of tasks ($n \gg k-1$):

$$S = \frac{t_n}{t_p}$$

- If the time to process a task is the same ($t_n = kt_p$):

$$S = \frac{kt_p}{t_p} = k$$

Example

Assume a pipeline has 4 segments with delays of 60, 50, 90, and 80 nanoseconds, and the register delay is 10 nanoseconds. Calculate the maximum speedup of the pipelined execution compared to the non-pipelined execution.

Pipeline Efficiency

$$\text{Efficiency} = \frac{\text{Speedup}}{\text{Number of Pipeline Stages}} \times 100\%$$

This formula is a rough approximation and assumes that all pipeline stages are fully and continuously utilized.

✓ Calculate the efficiency in the previous example.

Example

How many clock cycles does it take to execute 100 tasks on an 8-stage pipeline if:

a) The tasks enter the pipeline one after another (normally)

b) The tasks enter in separate batches of 20

c) The tasks enter in separate batches of 40, 20, 20, 10, 10

d) The tasks enter in batches of 40, 20, 10, 10, 10, 10

Example

If *n* instructions enter a *k*-stage pipeline in *m* separate batches, what is the total execution time?"

Pipelining

- To perform a multiply and add operation with a stream of numbers in a **pipelined** manner:
  - $A_i * B_i + C_i$ $\quad for\ i = 1, 2, 3, \ldots, 7$

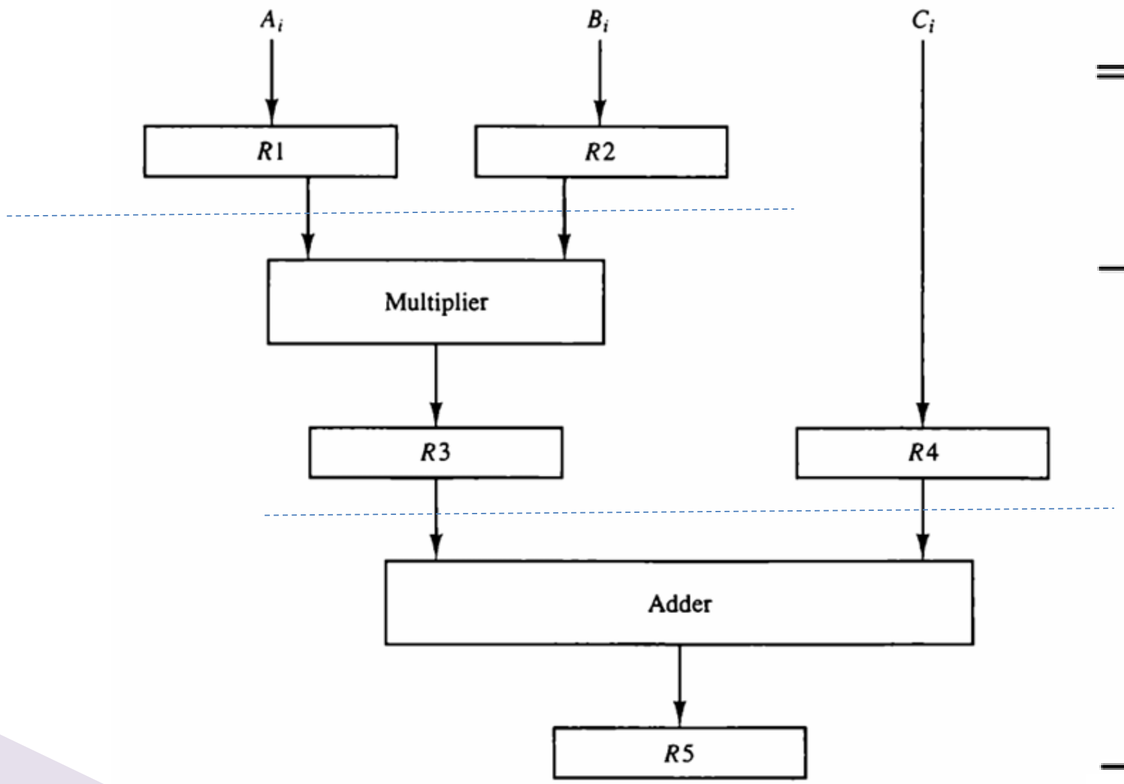


TABLE 9-1 Content of Registers in Pipeline Example

Clock Pulse Number	Segment 1		Segment 2		Segment 3
	R1	R2	R3	R4	R5
1	A_1	B_1	—	—	—
2	A_2	B_2	$A_1 * B_1$	C_1	—
3	A_3	B_3	$A_2 * B_2$	C_2	$A_1 * B_1 + C_1$
4	A_4	B_4	$A_3 * B_3$	C_3	$A_2 * B_2 + C_2$
5	A_5	B_5	$A_4 * B_4$	C_4	$A_3 * B_3 + C_3$
6	A_6	B_6	$A_5 * B_5$	C_5	$A_4 * B_4 + C_4$
7	A_7	B_7	$A_6 * B_6$	C_6	$A_5 * B_5 + C_5$
8	—	—	$A_7 * B_7$	C_7	$A_6 * B_6 + C_6$
9	—	—	—	—	$A_7 * B_7 + C_7$

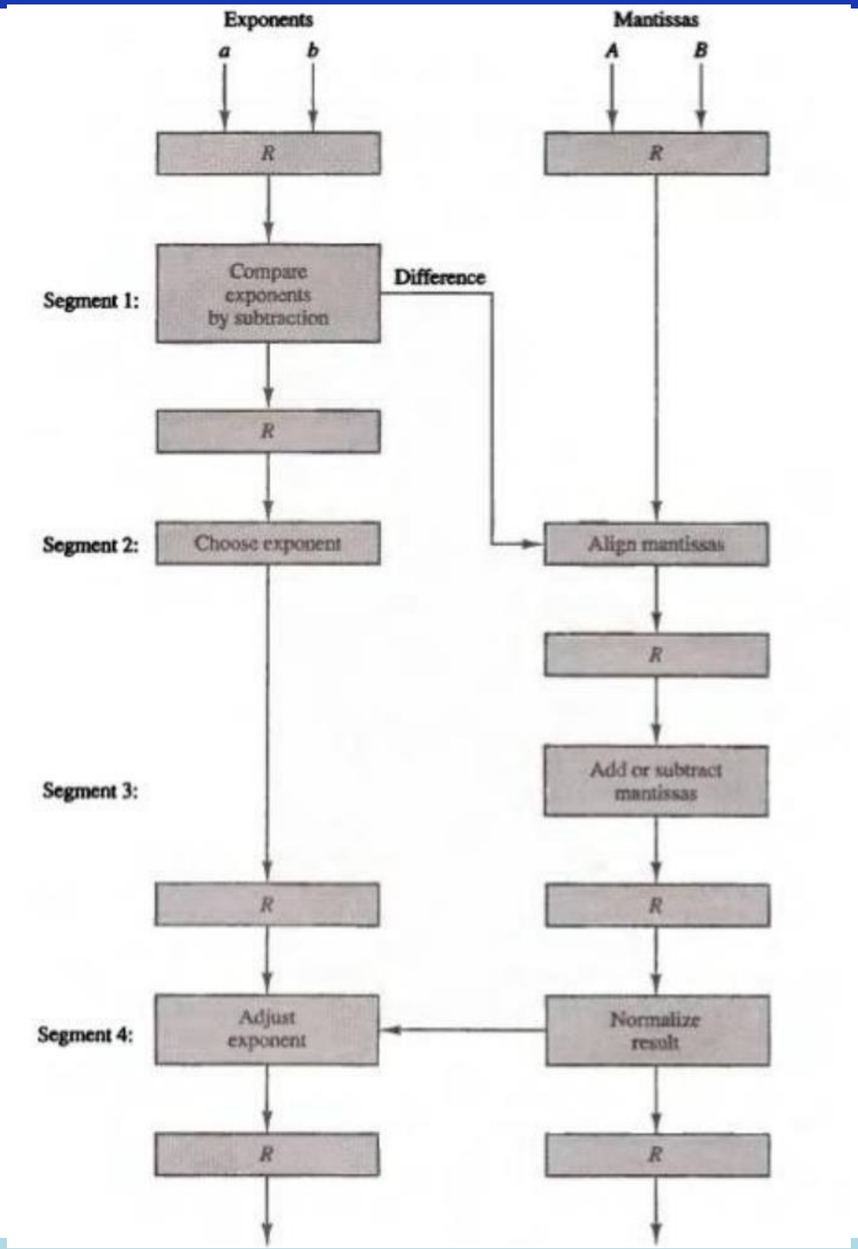
Arithmetic Pipeline

- Pipeline Arithmetic units are found in very high speed computers.
- They are used for implementing floating point operations, multiplication of fixed point numbers, … .

- Floating point operations are easily decomposed into suboperations.

- The sub operations of floating-point addition/subtraction:
    1. Compare the exponents
    2. Align the mantissas
    3. Add or subtract the mantissas
    4. Normalize the result

Pipeline for FP add/sub



Pipelined Addition example

- Consider two normalized floating-point numbers:

$$X = 0.9504 \times 10^3$$

$$Y = 0.8200 \times 10^2$$

- Segment 1: two exponents are subtracted (3-2=1). Larger exponent is chosen.
- Segment 2: shift the mantissa of Y to the right:

$$X = 0.9504 \times 10^3$$

$$Y = 0.0820 \times 10^3$$

- Segment 3: addition of the mantissas

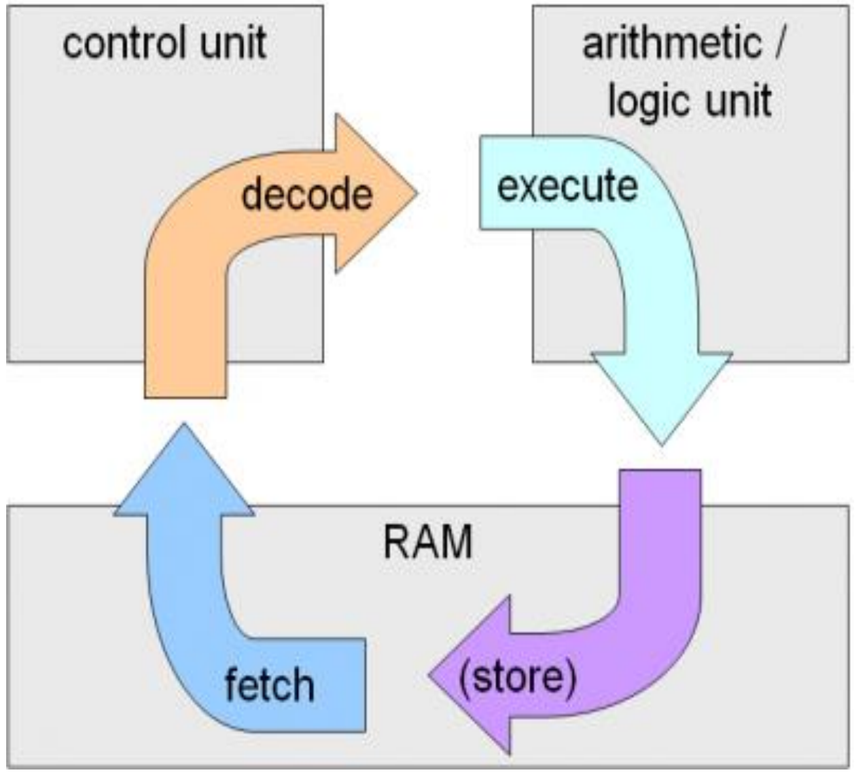
$$Z = 1.0324 \times 10^3$$

- Segment 4: Normalize the result

$$Z = 0.10324 \times 10^4$$

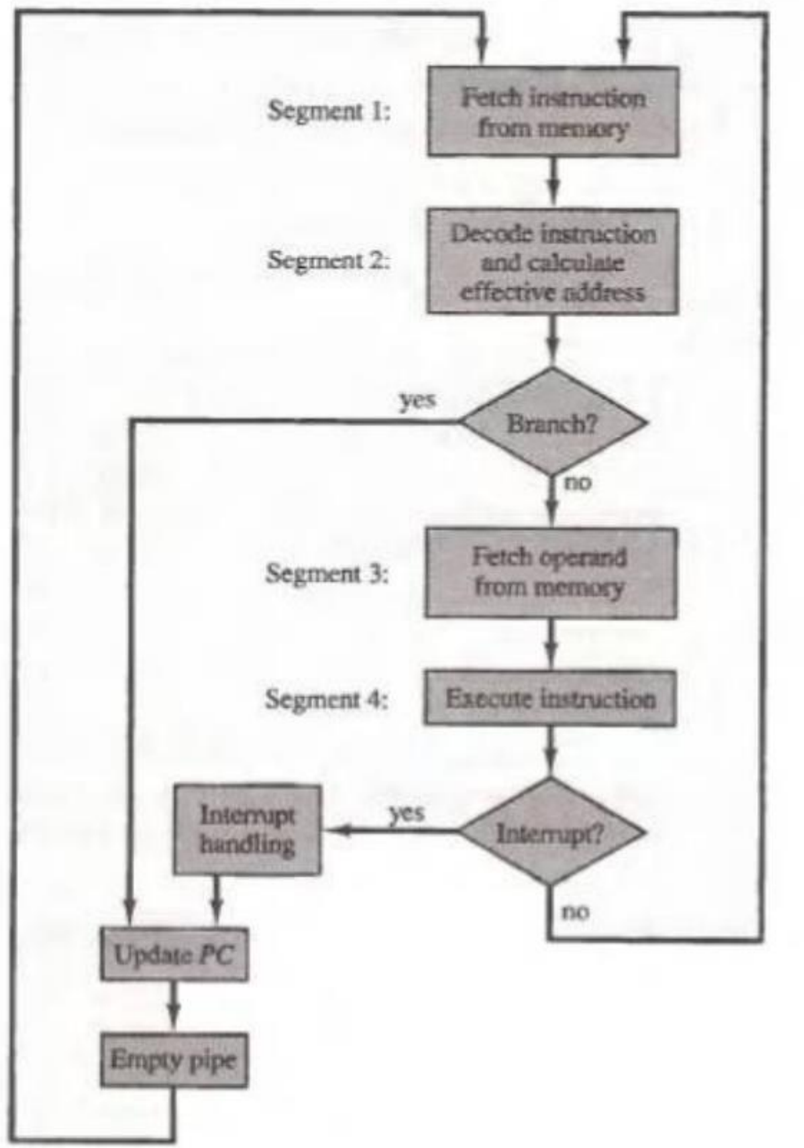
Instruction Pipelining

- Instructions in the most general case take five steps:
  1. Fetch the instruction from memory
  2. Decode the instruction
  3. Calculate the effective address.
  4. Fetch the operands from memory.
  5. Execute the instruction
  6. Store the result in the proper place.



Four-Segment Instruction Pipeline

- Assume that the decoding of instruction can be combined with the calculation of the effective address. This reduces the pipeline to four segments.
- Therefore, up to four suboperations in the instruction cycle can overlap and up to four different instructions can be in progress of being processed at the same time.



Four-Segment Instruction Pipeline

The following figure represents a demonstration of **Four-Segment Instruction Pipeline**.

Each row belongs to an **instruction** and each column is a **step** taken by CPU.

Each cell represents a segment as follows:

1. **FI**: Instruction Fetch
2. **DA**: Instruction Decode and Effective Address Calculation
3. **FO**: Operands Fetch
4. **EX**: Instruction Execution

	Step:	1	2	3	4	5	6	7	8	9	10	11	12	13
Instruction:	1	FI	DA	FO	EX									
	2		FI	DA	FO	EX								
(Branch)	3			FI	DA	FO	EX							
	4				FI	–	–	FI	DA	FO	EX			
	5					–	–	–	FI	DA	FO	EX		
	6									FI	DA	FO	EX	
	7										FI	DA	FO	EX

Pipeline Conflicts

Example: Assume now that instruction #3 is a branch instruction. As soon as the instruction is decoded in segment DA in step #4, the transfer from FI to DA of the other instructions **is halted** until the branch instruction is executed in step #6. If the branch is taken, another instruction is fetched from memory in step #7. Otherwise, the instruction fetched previously in step #4 can be used.

Step:		1	2	3	4	5	6	7	8	9	10	11	12	13
Instruction:	1	FI	DA	FO	EX									
	2		FI	DA	FO	EX								
(Branch)	3			FI	DA	FO	EX							
	4				FI	–	–	FI	DA	FO	EX			
	5					–	–	–	FI	DA	FO	EX		
	6									FI	DA	FO	EX	
	7										FI	DA	FO	EX

Pipeline Conflicts

- In general, there are three major difficulties that cause the pipeline to deviate from its normal operation:

1. **Resource Conflicts**

   Caused by access to memory by two segments at the same time. Most of these conflicts can be resolved by using separate instructions and data memories

2. **Data Dependency**

   Conflicts arise when and instruction depends on the result of a previous instruction, but this result is not yet available.

3. **Branch difficulties**

   Arise from branch and other instructions that change the value of PC.

Data Dependency

- **Data collision** occurs when an instruction cannot proceed because previous instructions did not complete certain operations.
- A **data dependency** occurs when an instruction needs data that are not yet available.
- **Example**:
  - An instruction in the FO segment may need to fetch an operand that is being generated at the same time by the previous instruction in segment EX.
  - An operand address cannot be calculated because the information needed by the addressing mode is not available.
  - An instruction with register indirect mode cannot proceed to fetch the operand if the previous instruction is loading the address into register

Conflict solutions

- **Hardware Interlocks**
  - An interlock is a circuit that detects instructions whose source operands are destinations of instructions farther up in the pipeline.
  - Detection causes the instruction whose resource is not available to be delayed by enough clock cycles to resolve the conflict.
  - This approach maintains the program sequence by using hardware to insert the required delays.

Conflict solutions

- **Operand Forwarding**
  - Uses special hardware to detect a conflict and then avoid it by routing the data through special paths between pipeline segments.
  - This method requires additional hardware paths through multiplexers as well as the circuit that detects the conflict.
  - **Example**:
    - Instead of transferring an ALU result into a destination register, the hardware checks the destination operand, and it is needed as a source in the next instruction, it passes the result directly intro the ALU input.

Conflict solutions

- **Delayed load**
  - Give the responsibility for solving data conflicts problems to the compiler that translates the high-level programming language into a machine language program.
  - The compiler is designed to detect a data conflict and reorder the instructions as necessary to delay the loading of the conflicting data by inserting no-operation instructions.

Handling of Branch Instructions

- The branch instruction breaks the normal sequence of the instruction stream, causing difficulties in the operation of the instruction pipeline.
- Pipelined computers employ various hardware techniques to minimize the performance degredation caused by instruction branching.
- **Prefetch target instruction**
  - prefetch the target instruction in addition to the instruction following the branch.
  - Both are saved until the branch is executed. If the branch condition is successful, the pipeline continues from the branch target instruction.
  - An extension of this procedure is to continue fetching instructions from both places until the branch decision is made. At that time control chooses the instruction stream of the correct program flow.

Handling of Branch Instructions

- **Branch target buffer**
  - branch target buffer or BTB is an associative memory included in the fetch segment of the pipeline.
  - Each entry in the BTB consists of the address of a previously executed branch instruction and the target instruction for that branch. It also stores the next few instructions after the branch target instruction.
  - When the pipeline decodes a branch instruction, it searches the associative memory BTB for the address of the instruction.
  - If it is in the BTB, the instruction is available directly, prefetch continues from the new path. If the instruction is not in the BTB, the pipeline shifts to a new instruction stream and stores the target instruction in the BTB.

Handling of Branch Instructions

- **Loop buffer**
  - A variation of the BTB is the loop buffer.
  - This is a small very high speed register file maintained by the instruction fetch segment of the pipeline.
  - When a program loop is detected in the program, it is stored in the loop buffer in its entirety, including all branches.
  - The program loop can be executed directly without having to access memory until the loop mode is removed by the final branching out.

Handling of Branch Instructions

- **Branch prediction**
  - Uses some additional logic to guess the outcome of a conditional branch instruction stream from the predicted path
  - A correct prediction eliminates the wasted time caused by branch penalties

- **Delayed Branch**
  - Employed in most RISC processors.
  - The compiler detects the branch instructions and rearranges the machine language code sequence by inserting useful instructions that keep the pipeline operating without interruptions.

Thanks